Laporan Sequential Pattern Mining pada Dataset Online Retail

Zico Diego Rio Ramadhonny (5054251023)

1. Ringkasan

Notebook ini menerapkan sequential pattern mining pada data transaksi toko online (UCI Online Retail) untuk menemukan urutan pembelian produk yang sering muncul pada pelanggan yang sama. Setelah pembersihan, 397.884 baris transaksi dari 4.338 pelanggan diubah menjadi urutan keranjang belanja berbasis StockCode, lalu ditambang dengan PrefixSpan dan GSP (SPMF) pada minimum support 1% (44 pelanggan).

Kedua algoritma menghasilkan himpunan pola yang identik: 194.580 pola dengan support yang sama persis. Pola berurutan teratas adalah membeli JUMBO BAG RED RETROSPOT lalu JUMBO BAG VINTAGE DOILY (5,44% pelanggan, 236 orang), disusul LUNCH BAG RED RETROSPOT lalu LUNCH BAG VINTAGE DOILY (4,91%) dan LUNCH BAG BLACK SKULL (4,89%). Perbedaan kedua algoritma hanya pada kinerja: PrefixSpan memerlukan sekitar 34 detik dan GSP sekitar 10 detik, sedangkan GSP memakai memori jauh lebih besar (sekitar 638 MB dibanding 24 MB). Kekurangan yang tersisa dibahas pada bagian 8.

2. Tujuan dan Konsep Dasar

Sequential pattern mining mencari urutan item yang sering muncul secara berurutan dalam waktu. Berbeda dengan association rule yang mengabaikan waktu, di sini urutan penting: membeli A lalu B pada kunjungan berikutnya berbeda dengan membeli B lalu A.

Istilah yang dipakai dalam notebook:

- **Item**: satu produk (diwakili oleh kolom Description).
- **Itemset/keranjang**: kumpulan produk yang dibeli pelanggan pada satu waktu transaksi.
- **Sequence**: daftar keranjang milik satu pelanggan, terurut berdasarkan waktu.
- **Support**: proporsi sequence (pelanggan) yang memuat suatu pola. Pola disebut frequent jika support-nya mencapai ambang minimum (minsup).

Tujuan praktisnya adalah memahami kebiasaan pembelian berulang pelanggan, misalnya sebagai dasar rekomendasi produk atau strategi promosi lanjutan.

3. Dataset

Dataset berasal dari UCI Machine Learning Repository (Online Retail) dan dibaca dari berkas Excel.

Aspek	Nilai
Jumlah baris awal	541.909
Jumlah kolom	8 (InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country)
Periode data	1 Desember 2010 – 9 Desember 2011
Nilai hilang	Description 1.454; CustomerID 135.080
Anomali	Quantity minimum −80.995 dan UnitPrice minimum −11.062,06 (retur/penyesuaian)

EDA awal sudah menunjukkan bahwa data perlu dibersihkan: ada kuantitas negatif, harga negatif, dan banyak transaksi tanpa identitas pelanggan.

4. Praproses Data

Tahapan praproses pada notebook:

1. **Menghapus baris tidak valid**: hanya baris dengan Quantity > 0, UnitPrice > 0, dan InvoiceNo yang tidak diawali huruf C (faktur pembatalan) yang dipertahankan, lalu baris dengan nilai kosong dihapus. Hasilnya 397.884 baris; 144.025 baris (26,6%) dibuang.
2. **Memilih fitur**: CustomerID, InvoiceDate, StockCode, dan Description. Item yang ditambang adalah StockCode; Description hanya dipakai sebagai label tampilan.
3. **Memperbaiki tipe data**: StockCode dan Description menjadi string, InvoiceDate menjadi datetime, CustomerID menjadi float.
4. **Membuat label per StockCode**: satu StockCode bisa memiliki beberapa Description, sehingga labelnya diambil dari Description yang paling sering muncul (setelah spasi ujung dibuang dan huruf dijadikan kapital).
5. **Pengelompokan**: baris dikelompokkan per (CustomerID, InvoiceDate) menjadi keranjang berupa tuple StockCode, lalu keranjang diurutkan menurut waktu dan dikumpulkan per CustomerID menjadi satu sequence.

5. Representasi Data untuk SPMF

SPMF hanya menerima item bilangan bulat, sehingga 3.665 StockCode unik dipetakan ke ID 1 sampai 3.665 menurut urutan StockCode yang sudah diurutkan, sehingga pemetaannya sama di setiap eksekusi. Setiap pelanggan ditulis sebagai satu baris: item dalam satu keranjang dipisah spasi dan diakhiri `-1`, lalu seluruh baris diakhiri `-2`. Contoh: `12 7 -1 5 -1 9 3 -1 -2`. SPMF mensyaratkan item dalam satu keranjang terurut menaik dan tanpa duplikat; syarat ini dipenuhi lewat `sorted({...})` sebelum penulisan.

Jumlah sequence sama dengan jumlah pelanggan, yaitu 4.338. Angka ini dicetak notebook sebagai Banyak sequence (4338) dan menjadi penyebut support; jumlah pelanggan per pola pada bagian 7.3 dihitung dari support dikali 4.338 (misalnya 0,0544 × 4.338 = 236). Minimum support 1% setara 44 sequence (0,01 × 4.338 = 43,38, dibulatkan ke atas).

6. Algoritma dan Parameter

Algoritma	Prinsip kerja	Parameter
PrefixSpan	Pattern-growth: memproyeksikan database berdasarkan prefix pola, tanpa membuat kandidat	minsup = 0,01
GSP	Generate-and-test: membangkitkan kandidat level demi level dan menghitung supportnya lewat pemindaian database	minsup = 0,01

Minsup 0,01 berarti sebuah pola harus muncul pada minimal 44 pelanggan. Memori Java dialokasikan 4.000 MB, dan SPMF membutuhkan Java terpasang (notebook mencatat Java 25). Kedua algoritma dijalankan lewat pustaka wrapper `spmf` pada berkas input yang sama.

7. Hasil

7.1 Statistik eksekusi

Metrik	PrefixSpan	GSP
Jumlah pola frequent	194.580	194.580
Waktu (internal SPMF)	34.458 ms	10.450 ms

Metrik	PrefixSpan	GSP
Memori maksimum (perkiraan SPMF)	24,3 MB	638,0 MB

Pada lima kali eksekusi dengan input yang sama, PrefixSpan konsisten di kisaran 34.065 sampai 34.852 ms dan GSP di kisaran 10.163 sampai 11.631 ms.

7.2 Verifikasi kesamaan hasil

Kedua algoritma menghasilkan 194.580 pola yang identik beserta supportnya. Uji pertama, yang membandingkan pola apa adanya, hanya mencocokkan 99.824 pola (51,3%) karena urutan item di dalam satu keranjang pada keluaran SPMF berbeda antar-algoritma (misalnya GSP menulis keranjang dengan urutan item yang tidak terurut). Setelah parser mengurutkan item dalam tiap keranjang, kedua himpunan sama persis. Perbedaan sebelumnya hanya soal penulisan, bukan perbedaan hasil.

Kondisi	Ukuran himpunan (PrefixSpan / GSP)	Irisan	Hanya di satu sisi
Item dalam keranjang belum diurutkan	194.580 / 194.580	99.824	94.756 per sisi
Item dalam keranjang diurutkan	194.580 / 194.580	194.580	0

7.3 Pola berurutan teratas

Pola ditampilkan hanya bila terdiri dari minimal dua keranjang (kunjungan berbeda) dan tidak memuat item yang berulang, sehingga pola satu item dan pembelian ulang item yang sama tidak ikut. Hasilnya identik pada kedua algoritma.

Peringkat	Pola (kunjungan awal → kunjungan berikutnya)	StockCode	Support	Pelanggan
1	JUMBO BAG RED RETROSPOT → JUMBO BAG VINTAGE DOILY	85099B → 23203	0,0544	236
2	LUNCH BAG RED RETROSPOT → LUNCH BAG VINTAGE DOILY	20725 → 23209	0,0491	213

Peringkat	Pola (kunjungan awal → kunjungan berikutnya)	StockCode	Support	Pelanggan
3	LUNCH BAG RED RETROSPOT → LUNCH BAG BLACK SKULL.	20725 → 20727	0,0489	212

7.4 Interpretasi

Ketiga pola berpasangan dalam keluarga produk yang sama: pelanggan yang membeli satu desain (Red Retrospot) cenderung membeli desain lain dari jenis produk yang sama (Jumbo Bag atau Lunch Bag) pada kunjungan berikutnya. Pola ini dimiliki sekitar 1 dari 18 sampai 20 pelanggan. Support hanya menyatakan proporsi pelanggan yang memuat pola, bukan peluang bersyarat, jadi belum bisa dibaca sebagai pelanggan yang membeli A pasti membeli B; untuk itu diperlukan confidence dari aturan sekuensial.

GSP sekitar 3,3 kali lebih cepat tetapi memakai memori sekitar 26 kali lebih besar. Hasil waktu ini konsisten pada lima eksekusi, namun biasanya PrefixSpan lebih cepat, dan penyebabnya belum dianalisis. Nilai memori dari SPMF hanya perkiraan kasar.

8. Masalah yang Diperbaiki dan Kekurangan yang Tersisa

Notebook versi ini sudah memperbaiki masalah utama dari versi sebelumnya. Tabel berikut merangkum perbaikannya, lalu dibahas kekurangan yang masih tersisa.

8.1 Masalah yang sudah diperbaiki

Masalah di versi sebelumnya	Dampak	Perbaikan
Item dalam keranjang tidak terurut dan tidak bebas duplikat saat ditulis untuk SPMF	Jumlah pola berbeda jauh (59.216 vs 141.507)	Setiap keranjang diurutkan dan dideduplikasi sebelum ditulis; jumlah pola kini sama (194.580)
Urutan item dalam keranjang pada keluaran SPMF berbeda antar-algoritma	Uji kesamaan bernilai False (irisan hanya 99.824 pola)	Parser mengurutkan item dalam tiap keranjang; uji kesamaan bernilai True
Penanda -1 dibuang saat parsing	Pola (A B) tidak bisa dibedakan dari A → B	Parser menyimpan tiap keranjang sebagai tuple

Masalah di versi sebelumnya	Dampak	Perbaikan
Description dipakai sebagai item	Variasi nama produk memecah support	Item adalah StockCode; label diambil dari Description yang paling sering muncul
Label per StockCode diambil dari kemunculan terakhir	Produk berlabel salah (85123A tampil sebagai CREAM)	Label memakai modus Description
Baris dengan UnitPrice ⩽ 0 tidak disaring	Baris tidak valid ikut diproses	Disaring (40 baris)
Pola teratas hanya satu item atau pembelian ulang item yang sama	Tidak ada informasi sekuensial	Hanya pola minimal dua keranjang dan tanpa item berulang yang ditampilkan
ID item dibuat dari iterasi set	Pemetaan bisa berubah antar-eksekusi	ID dibuat dari StockCode yang sudah diurutkan

8.2 Kekurangan yang tersisa

Penyaringan item non-produk dan pengujian sensitivitas minsup tidak diminta pada praktikum ini, sehingga tidak dihitung sebagai kekurangan. Yang tersisa:

1. **Label masih memuat spasi ganda.** Contohnya LUNCH BAG BLACK SKULL. pada pola ketiga, karena hanya spasi di ujung yang dibuang.
2. **Definisi keranjang berbasis InvoiceDate.** Pengelompokan memakai stempel waktu hingga menit, bukan InvoiceNo. Dua faktur pada menit yang sama tergabung, sedangkan faktur di hari yang sama pada menit berbeda dianggap kunjungan berbeda. Pilihan ini belum dijelaskan.
3. **Filter hasil membuang pola bermakna.** Syarat minimal dua keranjang dan tanpa item berulang menyingkirkan pola bundel (beberapa item dalam satu kunjungan) dan pembelian ulang item yang sama. Keduanya informatif, dan keputusan menyaringnya belum dijelaskan.
4. **Penyebab PrefixSpan lebih lambat belum dianalisis.** PrefixSpan konsisten lebih lambat daripada GSP pada lima eksekusi, padahal biasanya sebaliknya. Waktu dan memori hanya dicetak oleh SPMF dan belum dikumpulkan dalam tabel di notebook.
5. **Kerapian.** Konversi tipe memakai zip terhadap urutan kolom dan map(np.datetime64) yang lambat dibanding pd.to_datetime; keluaran memuat path lokal pribadi dan log peringatan Java.

6. **Narasi minim.** Notebook memiliki judul tiap tahap, tetapi belum ada penjelasan tujuan, interpretasi hasil, maupun kesimpulan.

9. Rekomendasi Perbaikan

1. Rapikan label dengan menghilangkan spasi ganda (contoh kode di bawah).
2. Jelaskan alasan memilih InvoiceDate sebagai batas keranjang, atau ganti dengan InvoiceNo.
3. Jelaskan alasan menyaring pola minimal dua keranjang tanpa item berulang, atau tampilkan juga pola bundel dan pembelian ulang.
4. Kumpulkan waktu dan memori kedua algoritma dalam satu tabel perbandingan, dan telaah mengapa PrefixSpan lebih lambat pada data ini.
5. Tambahkan sel markdown berisi tujuan, interpretasi, dan kesimpulan pada setiap tahap.

10. Kesimpulan

Notebook kini menghasilkan keluaran yang valid dan konsisten: format input sesuai SPMF, struktur keranjang terjaga saat parsing, dan PrefixSpan serta GSP menghasilkan 194.580 pola yang identik dengan support yang sama. Pola berurutan teratas menunjukkan pelanggan yang membeli satu desain Jumbo Bag atau Lunch Bag Red Retrospot cenderung membeli desain lain dari jenis yang sama pada kunjungan berikutnya, dengan support 4,9% sampai 5,4%.

Kekurangan yang tersisa tidak mengubah kebenaran hasil: label masih memuat spasi ganda, pilihan batas keranjang dan filter pola belum dijelaskan, dan interpretasi tertulis belum ada di notebook. Setelah rekomendasi pada bagian 9 diterapkan, notebook akan lebih mudah dipahami dan dipertanggungjawabkan.